

MAS Final Project: Pathfinding and Cooperative Planner (Complete Code Documentation)

1. Scope

This chapter documents `pathfinding.py` in full detail.

Coverage includes:

- all module constants,
- all data structures,
- all helper functions,
- all planner classes and methods,
- full `compute_actions(...)` execution flow,
- interfaces with `env.py`, `agent.py`, `config.py`, `verification.py`, and `refinement.py`.

2. Imports and Type Aliases

`pathfinding.py` imports:

- `defaultdict` from `collections`
- `dataclass` from `dataclasses`
- `heapq`
- typing primitives (`Dict`, `Iterable`, `List`, `Optional`, `Set`, `Tuple`)
- `Action`, `Direction`, `Robot` from `agent.py`

Type alias:

- `GridPos = Tuple[int, int]` for cell coordinates (x, y).

3. Module-Level Motion and Cost Constants

3.1 Directional Forward Deltas

- `FORWARD_DX = (0, 1, 0, -1)`
- `FORWARD_DY = (-1, 0, 1, 0)`

Indexing uses `Direction.value`:

- 0=UP, 1=RIGHT, 2=DOWN, 3=LEFT

3.2 Action Expansion Order

- `ACTION_ORDER = (FORWARD, TURN_LEFT, TURN_RIGHT, WAIT)`

This controls:

- A* neighbor expansion ordering,
- immediate fallback action evaluation order.

3.3 A* Transition Costs

- FORWARD: 1.0
- TURN_LEFT: 1.0
- TURN_RIGHT: 1.0
- WAIT: 1.25

WAIT is deliberately more expensive than movement/turn to discourage idle plans.

3.4 Fallback Scoring Constants

When no full path is available, immediate action scoring uses:

- NO_TARGET_SCORES when target is None
- TARGET_PENALTIES when target exists

These bias behavior toward motion over passive waiting and penalize unnecessary turns.

4. Node Representation for Space-Time Search

@dataclass(frozen=True) class NodeKey fields:

- x: int
- y: int
- dir_value: int
- t: int

Used as hashable keys for:

- g_score,
- parent predecessor map,
- open-list bookkeeping.

frozen=True ensures immutability and safe dictionary usage.

5. Timed Occupancy Data Structures

5.1 _TimedTable

Base class with two sparse time-indexed maps:

- positions: Dict[int, Set[GridPos]]
- edges: Dict[int, Set[Tuple[GridPos, GridPos]]]

Methods:

- has_position(pos, t)
- has_edge(from_pos, to_pos, t)

Provides generic storage/query reused by reservation and constraint systems.

5.2 ReservationTable

Purpose:

- reserve already-planned occupancy for conflict-free sequential planning.

Methods:

- `is_reserved(pos, t)` -> vertex reservation check
- `is_edge_reserved(from_pos, to_pos, t)` -> directed edge reservation check
- `reserve_position(pos, t)`
- `reserve_edge(from_pos, to_pos, t)`
- `reserve_positions(positions)`

`reserve_positions(...)` behavior:

- reserves each position at each time index,
- for $t > 0$, reserves directed transition edge when position changed.

This supports both vertex and edge conflict prevention.

5.3 ConstraintTable

Purpose:

- hold externally-added hard constraints from refinement/verification.

Methods:

- `forbid_position(pos, t)`
- `forbid_edge(from_pos, to_pos, t)`
- `is_forbidden(pos, t)`
- `is_edge_forbidden(from_pos, to_pos, t)`

Unlike reservations (ephemeral per step), these are persistent planner constraints.

6. Utility Functions

6.1 `manhattan(a, b)`

Returns Manhattan distance:

- $\text{abs}(ax - bx) + \text{abs}(ay - by)$

Used for:

- A* heuristic,
- assignment costs,
- delivery slot costs,
- immediate scoring.

6.2 `minimum_cost_matching(costs)`

Implements Hungarian algorithm for rectangular cost matrices.

Input:

- `costs[i][j] = cost(row_i -> col_j)`

Output:

- list of selected column indices per row (-1 means unassigned).

Implementation details:

1. Handles empty rows/columns.
2. Pads to square matrix with large `pad_cost = max_cost + 1e6`.
3. Uses standard Hungarian dual variables:
 - `u`, `v` (potentials),
 - `p` (matching),
 - `way` (augmenting path reconstruction).
4. Reconstructs row assignment for original dimensions only.

Used by:

- robot->shelf assignment,
- carrier->goal slot assignment.

6.3 `apply_action(...)`

Signature:

- `(x, y, direction, action, grid_w, grid_h) -> Optional[(nx, ny, ndir)]`

Behavior:

- `TURN_LEFT`: rotate in place.
- `TURN_RIGHT`: rotate in place.
- `WAIT` or `PICK_DROP`: no motion, same direction.
- `FORWARD`: move one cell in facing direction.
- out-of-bounds forward -> returns `None`.
- unknown action -> raises `ValueError`.

This is the common deterministic transition model used by planner and simulation helpers.

6.4 `simulate_positions(...)`

Rolls out actions from `(start_pos, start_dir)` for fixed horizon.

Behavior:

- if action list shorter than horizon, fills remaining with `WAIT`,

- invalid forward transition (None) keeps prior state,
- returns positions list length `horizon + 1` including initial position.

Used to convert chosen action sequence into reservation window occupancy.

6.5 `_blocked_by_constraints(...)`

Checks optional list of `ConstraintTables` for:

- forbidden next vertex at time `t`,
- forbidden directed edge when movement occurs.

Returns boolean block decision.

7. Space-Time A* (`astar_time`)

7.1 Signature and Inputs

```
astar_time(start_pos, start_dir, goal, grid_w, grid_h, reservations,
max_time, blocked_t1, blocked_by_time=None, blocked_static=None,
visual=None, constraints=None, max_expansions=6000)
```

Key inputs:

- `reservations`: prior-planned occupancy,
- `blocked_t1`: immediate blocked cells at `t=1`,
- `blocked_by_time`: conservative future blocked map,
- `blocked_static`: static blocked cells (usually shelves),
- `constraints`: hard forbidden tables,
- `max_expansions`: node expansion cap.

`visual` is accepted but currently unused.

7.2 Open List and Scores

Open list tuple:

- `(f, g, sequence, NodeKey)`

`sequence` breaks ties deterministically for heap stability.

State records:

- `g_score`: `Dict[NodeKey, float]`
- `parent`: `Dict[NodeKey, (prev_node, action)]`

7.3 Goal and Termination

Algorithm stops and reconstructs path when current `(x, y)` equals goal position.

Goal does not require particular direction.

Other terminations:

- expansions exceed `max_expansions` -> `None`
- no reachable nodes -> `None`

7.4 Transition Filtering

For each action in `ACTION_ORDER`, candidate transition is rejected if any condition holds:

1. invalid action transition (`None`)
2. target in `blocked_static`
3. time-1 target in `blocked_t1`
4. target in `blocked_by_time[nt]`
5. target vertex reserved in `reservations`
6. edge conflict with reservation in either direction
7. forbidden by constraint table
8. no g-score improvement

Accepted candidates are pushed with:

- `f = next_g + manhattan(next_pos, goal)`
- parent pointer for reconstruction.

8. AssignmentManager

State maps:

- `agent_to_shelf: Dict[int, int]`
- `shelf_to_agent: Dict[int, int]`

8.1 update_assignments(env)

Flow:

1. Build `shelves_by_id`.
2. Prune stale assignments where shelf no longer valid:
 - must still exist,
 - not carried,
 - still requested.
3. Build free robot set:
 - not carrying,
 - currently unassigned.
4. Build free shelf set:
 - requested,
 - not carried,
 - unassigned.
5. Build Manhattan cost matrix.
6. Solve with Hungarian matching.

7. Commit consistent two-way maps.

8.2 `get_target_for_robot(robot, env, delivery_targets=None)`

Cases:

1. Robot carrying requested shelf:
 - use `delivery_targets[robot.id]` if available,
 - else nearest goal via `_nearest_goal`.
2. Robot carrying non-requested shelf:
 - no target (`None`).
3. Robot empty:
 - use assigned shelf if still valid.
 - if assignment invalid, remove mapping and return `None`.

8.3 `_nearest_goal(pos, goals)`

- raises `ValueError` if no goals.
- otherwise returns goal minimizing Manhattan distance.

9. `IdleTracker`

State:

- `idle_limit`
- `last_positions`
- `idle_steps`

Method `track_idle_agents(env, assignment_manager)`:

1. Compare current vs previous position.
2. Increment or reset idle counter.
3. If idle count reaches limit:
 - drop robot assignment from manager maps,
 - reset robot idle count.

Purpose:

- reduce persistent assignment deadlocks/starvation.

10. `CooperativePlanner` Overview

This is the top-level planner class used by runtime and verification.

10.1 Constructor

`CooperativePlanner(grid_w, grid_h, plan_horizon=30)`

Initial validation:

- raises `ValueError` if non-positive grid dimensions.

Initial components:

- `self.constraints = ConstraintTable()`
- `self.assignment_manager = AssignmentManager()`
- `self.idle_tracker = IdleTracker()`
- `self.priority_offset = 0`
- `self.reservation_window = clamp(2..min(plan_horizon,8))`
- `self.unplanned_hold_steps = 2`
- `self.escape_idle_steps = 6`

Config override via `import config as cfg`:

- `ASTAR_MAX_NODES`
- `IDLE_LIMIT`
- `RESERVATION_WINDOW`
- `UNPLANNED_HOLD_STEPS`
- `ESCAPE_IDLE_STEPS`

Fallback values are used on `ImportError`.

10.2 Public Constraint Injection API

- `add_constraint_position(pos, t)`
- `add_constraint_edge(from_pos, to_pos, t)`

Both reject negative time with `ValueError`.

Used by refinement workflow.

11. Planning Order and Fairness

11.1 `_planning_order(robots)`

Mechanism:

1. Rotate list by `priority_offset` (round-robin fairness).
2. Increment `priority_offset` each call.
3. Sort rotated list by tuple:
 - carrying requested shelf first,
 - higher idle steps first,
 - rotated index tie-break.

Effects:

- delivery tasks prioritized,
- stalled robots get chance to move,
- global order is not permanently fixed.

11.2 `_base_debug(robot_id, priority)`

Creates debug payload with fields:

- `priority`
- `assigned shelf id`
- `idle steps`
- `target`
- `mode`
- `astar_found`
- `path_len`
- `path_preview`

Later augmented with chosen action and sanitize flags.

12. Reservation Commit Helpers

12.1 `_reserve_actions(...)`

Simulates positions for `reservation_window` horizon using:

- provided `planned_path`, or fallback chosen action,
- robot start pose,
- grid bounds.

Then reserves resulting positions/edges.

12.2 `_commit_plan(...)`

Writes chosen action and debug into:

- `actions_by_id`
- `planner_debug_by_agent`

and applies reservation through `_reserve_actions(...)`.

13. Navigation Planning (`_plan_navigation`)

Inputs include:

- `robot/env/reservations`,
- `blocked-by-time maps`,
- `shelf positions`,
- `allowed shelf entries map`,
- `delivery target map`,
- `mutable debug record`.

Flow:

1. Resolve target via `AssignmentManager`.
2. Build:

- `blocked_t1`,
 - `blocked_static` shelf cells via `_blocked_shelf_positions`.
3. Update debug block statistics.
 4. If empty robot targeting shelf cell, register in `allowed_shelf_entries`.

13.1 No Target Branch

If `target` is `None`:

- if robot empty and standing on goal:
 - choose clearing action toward nearest requested shelf via `_best_immediate_action`.
 - mode `goal_clear`.
- else:
 - wait with mode `no_target_wait`.

13.2 Targeted Branch

Runs `astar_time(...)` with:

- plan horizon,
- reservations,
- blocked maps,
- static shelf blocks,
- hard constraints list.

Result handling:

1. If path found:
 - mode `astar_path`,
 - return first action.
 - exception: if first is `WAIT`, target not reached, and robot very idle:
 - attempt escape action,
 - mode `stuck_path_escape`.
2. If no path and already at target:
 - wait, mode `target_reached_wait`.
3. If no path and idle beyond threshold:
 - attempt escape action,
 - mode `no_path_escape` if success.
4. Otherwise:
 - wait, mode `no_spacetime_path_wait`.

14. Main Planner Entry (`compute_actions`)

`compute_actions(env) -> List[int]`

Detailed step sequence:

1. Update idle tracking and assignments.

2. Create fresh `ReservationTable`.
3. Compute currently occupied positions.
4. Initialize action/debug maps and delivery targets.
5. Determine planning order and priority ranks.
6. Build set of non-carried shelf positions.
7. For each robot in planning order:
 - build immediate blocked set from occupied-now except self,
 - build conservative future blocked map for not-yet-planned robots up to `unplanned_hold_steps`,
 - create base debug record,
 - choose action:
 - if `_should_pick_drop(...)` true -> `PICK_DROP`,
 - else `_plan_navigation(...)`,
 - commit action/debug/reservations.
8. Publish planner metadata to env:
 - `_planner_allowed_shelf_entries`
 - `_planner_last_actions` (after sanitize)
 - `_planner_debug_by_agent`
9. Build ordered action list aligned with `env.robots`.
10. Sanitize immediate conflicts with `_sanitize_immediate_conflicts(...)`.
11. Add post-sanitize debug fields:
 - `chosen_action`
 - `sanitized` bool.
12. Return sanitized action integers.

15. Shelf Blocking Helper

`_blocked_shelf_positions(robot, target, env):`

- returns set of non-carried shelf cells treated as blocked for this robot,
- exception: empty robot may enter its own target shelf cell.

This mirrors execution-time shelf blocking policy in `agent.py`.

16. Goal and Delivery Helpers

16.1 `_nearest_requested_shelf(robot, env)`

- returns nearest requested uncarried shelf position or `None`.

16.2 `_assign_delivery_goals(env)`

Purpose:

- assign carrying requested robots to goal slots.

Method:

1. Collect carrier robots and goals.
2. Build repeated goal slots by `tier`:
 - each tier adds cost penalty $2.0 * tier$.
3. Build cost matrix:
 - Manhattan distance + tier penalty.
4. Hungarian matching gives carrier->slot.
5. Return `robot_id` -> `goal_position`.

This allows multiple carriers to share finite goals with staggered preference.

17. Immediate Conflict Sanitization

`_sanitize_immediate_conflicts(env, actions, priority_rank)`

Purpose:

- final safety net for first-step action consistency.

Flow:

1. Convert action ints to `Action` by robot id.
2. For robots in priority order, inspect forward intents:
 - invalid forward -> set WAIT,
 - forward into blocked shelf position -> set WAIT,
 - else record intent (`from`, `to`).
3. Resolve same-target conflicts:
 - first owner by priority keeps target,
 - others become WAIT.
4. Resolve edge swaps among active forward intents:
 - lower-priority conflicting robot becomes WAIT.
5. Return sanitized action list in env robot order.

18. Immediate Fallback Action Selection

`18.1 _best_immediate_action(robot, target, reservations, blocked_t1)`

Evaluates `ACTION_ORDER` candidates and rejects invalid/blocked/reserved/forbidden moves.

For each valid candidate:

- compute score via `_immediate_action_score(...)`,
- choose minimum score action.

Returns best action or WAIT default.

18.2 `_immediate_action_score(action, cur_pos, next_pos, target)`

If target is None:

- returns `NO_TARGET_SCORES[action]`.

If target exists:

- `manhattan(next_pos, target) + TARGET_PENALTIES[action] + stay_penalty`
- stay penalty is 0.2 if `next_pos == cur_pos`.

Encodes preference for progressing toward target and avoiding idleness.

19. Pick/Drop Trigger Logic

`_should_pick_drop(robot, env):`

Cases:

1. Robot carrying requested shelf:
 - return true iff robot is on a goal cell (deliver action).
2. Robot carrying non-requested shelf:
 - false.
3. Robot empty:
 - return true iff robot stands on:
 - uncarried requested shelf,
 - that shelf is assigned to this robot in assignment manager.

This prevents opportunistic wrong-robot pickups.

20. Planner Debug Contract

Per-agent debug records (stored in `env._planner_debug_by_agent`) include:

- priority
- assigned shelf id
- idle_steps
- target
- mode
- astar_found
- path_len
- path_preview
- chosen_action_pre_sanitize
- chosen_action
- sanitized
- block counts (`blocked_t1_count`, `blocked_future_count`, `blocked_static_count`)

These fields support diagnosis and UI introspection.

21. Cross-Module Interfaces

21.1 Input Expectations from Environment

Planner expects `env` with:

- `robots`, each with `id`, `x`, `y`, `dir`, `carrying`
- `shelves` dict list with keys:
 - `id`, `x`, `y`, `carried`, `requested`
- `GOALS`

21.2 Outputs Written to Environment

Planner writes:

- `env._planner_allowed_shelf_entries`
- `env._planner_last_actions`
- `env._planner_debug_by_agent`

21.3 Constraint Hooks for Refinement

Refinement uses:

- `add_constraint_position(...)`
- `add_constraint_edge(...)`

which feed directly into A* transition filtering.

22. Complexity and Runtime Behavior

Per timestep rough costs:

1. Assignment:
 - Hungarian complexity cubic in $\min(\text{robot_count}, \text{shelf_count})$ scale.
2. Planning:
 - one bounded A* per robot (`astar_max_nodes` cap).
3. Sanitization:
 - near $O(k^2)$ in active forward intents.

Practical stability controls:

- `astar_max_nodes`
- `plan_horizon`
- `reservation_window`
- conservative blocking for unplanned robots.

23. Determinism Notes

Given fixed:

- environment seed,

- configuration,
- robot ordering,
- tie-breaking sequence behavior,

planner behavior is deterministic.

Deterministic elements include:

- explicit action order,
- heap sequence tie-break,
- sorted robot iteration in key places.

24. Failure Modes and Tradeoffs

1. Prioritized planning can miss globally optimal joint plans.
2. Conservative future blocking may over-restrict paths.
3. Escape fallback is local heuristic, not guaranteed deadlock resolution.
4. Heavy congestion can trigger frequent waits if horizons/node caps are tight.
5. Persistent constraints may over-constrain unless managed.

25. Full Symbol Checklist (Module Coverage)

This chapter covered all top-level symbols in `pathfinding.py`:

- `GridPos`
- `FORWARD_DX`, `FORWARD_DY`
- `ACTION_ORDER`, `ACTION_COSTS`
- `NO_TARGET_SCORES`, `TARGET_PENALTIES`
- `NodeKey`
- `_TimedTable`
- `ReservationTable`
- `ConstraintTable`
- `manhattan`
- `minimum_cost_matching`
- `apply_action`
- `simulate_positions`
- `_blocked_by_constraints`
- `astar_time`
- `AssignmentManager`
- `IdleTracker`
- `CooperativePlanner` and all of its methods.

26. Chapter Summary

`pathfinding.py` is the decision core of the project.

It combines assignment, bounded space-time search, reservations, hard constraints, and final conflict sanitization into one deterministic multi-agent action

generator.

Verification and refinement integrate through constraint hooks, making the planner both operational and improvable under formal safety feedback.